Does Dimensionality Reduction Help Classical Intrusion Detection? A Leakage-Controlled Comparison of Original Features, PCA and Kernel PCA on a CIC-IDS2017 Sample

[Author One]
[Department], [Institution]
[City, Country]
[email]

[Author Two]
[Department], [Institution]
[City, Country]
[email]

October 6, 2026

Abstract

Kernel PCA (KPCA) is often paired with classical classifiers for network intrusion detection, but reported gains are usually measured with accuracy on a single split and without a no-reduction baseline. We compare three feature spaces—the original 78 flow features, PCA, and KPCA with linear, RBF and cosine kernels (5–30 components)—across five classifiers (logistic regression, k-NN, SVM, decision tree, random forest) on a 1,000-record stratified sample of CIC-IDS2017 (11 classes, 71.8% benign). Imputation, scaling and reduction are fitted on training folds only; evaluation uses repeated stratified 5-fold cross-validation (15 folds) with accuracy, macro-F1, balanced accuracy and MCC, paired Wilcoxon tests with Holm correction, and a runtime/memory benchmark. The strongest model was a random forest on the *original* features (accuracy 0.952 ± 0.010, macro-F1 0.722 ± 0.058, MCC 0.894 ± 0.022; majority-class baseline: accuracy 0.718, macro-F1 0.076). No PCA or KPCA setting improved on the original features for logistic regression, decision tree or random forest, and k-NN showed no significant difference; the exception was an RBF-kernel SVM, whose macro-F1 rose from 0.39 to 0.51 with 30-component cosine KPCA. Linear KPCA reproduced PCA, as theory predicts. KPCA raised fitting time roughly tenfold and peak traced memory from 1.7 to 11.7 MB at 800 training records, with memory growing quadratically in sample size. Results come from one small sample of one dataset; they do not establish generalization to unseen traffic.

***Index Terms*—** network intrusion detection, kernel PCA, dimensionality reduction, CIC-IDS2017, class imbalance, empirical comparison

I Introduction

Machine-learning network intrusion detection systems (NIDS) are commonly developed and compared on public flow datasets [1], of which CIC-IDS2017 [2] is one of the most used. Its flow records carry 78 numeric features, which motivates dimensionality reduction as a way to cut training cost and possibly improve generalization. Linear PCA and auto-encoders have been applied to CIC-IDS2017 [6], and Kernel PCA (KPCA) [7], which performs PCA in an implicit nonlinear feature space, has been combined with SVMs for intrusion detection [8].

Three methodological issues make such comparisons hard to interpret. First, CIC-IDS2017 is class-imbalanced and, in its original form, contains labelling and flow-construction problems [3, 4], so accuracy can be dominated by the benign majority. Second, common pitfalls in security-oriented machine learning—including evaluation set-ups that overstate real performance—are widespread [5]. Third, many studies omit the simplest baseline, the original feature space, so a benefit of the reduction is assumed rather than measured.

Research questions. We ask: **RQ1** how does reduction affect multi-class detection performance; **RQ2** does KPCA beat the original space or conventional PCA; **RQ3** how do kernel, component count and classifier interact; **RQ4** what does the reduction cost in time and memory; **RQ5** how stable are the conclusions under repeated, class-aware validation and smaller training sets.

Contributions. (i) A leakage-controlled, reproducible comparison of original features, PCA and three KPCA kernels over five classical classifiers, including a majority-class baseline. (ii) A class-aware analysis (macro-F1, balanced accuracy, MCC, per-class recall) showing how much accuracy overstates detection quality on this data. (iii) Paired statistical tests of the comparisons that matter, with an explicit caveat on their independence. (iv) A runtime and memory measurement of KPCA against PCA. We do not propose a new algorithm. All code, raw per-fold results and scripts are in the project repository.

II Related Work

Datasets. Ring et al. survey network-based intrusion datasets and the properties that determine their suitability for evaluation scenarios [1]. CIC-IDS2017 was introduced by Sharafaldin et al. [2]. Engelen et al. report problems in it: "TCP appendix"

artefact flows (about a quarter of all flows), attack attempts labelled as attacks although they contain no malicious payload, and feature-extraction flaws that enable shortcut learning; they released a corrected version [3]. Panigrahi and Borah likewise describe shortcomings of the dataset and propose a combined dataset [4]. We use the *original* labelled flows, so absolute scores here may partly reflect such artefacts.

Dimensionality reduction for NIDS. Abdulhammed et al. apply an auto-encoder and PCA to CIC-IDS2017, reducing the feature count from 81 to 10 while reporting 99.6% accuracy with random forest and other classifiers [6]. Kuang et al. propose a hybrid of KPCA, an SVM and a genetic algorithm for intrusion detection [8]. This section is not a systematic survey; it positions the present study rather than claiming to cover the literature.

Evaluation practice. Arp et al. document recurring methodological pitfalls in machine learning for security that lead to unrealistic performance [5], and Apruzzese et al. show that cross-dataset evaluation of NIDS reveals behaviour that single-dataset testing hides [9]. Motivated by this, we (a) fit every data-dependent step on training folds only, (b) report class-aware metrics next to accuracy, (c) include trivial and no-reduction baselines, and (d) state plainly what our sample cannot show.

III Dataset and Preprocessing

Data. The study uses the cached 1,000-record stratified sample shipped with the project (`data/sample.npz`), drawn from the Tuesday, Wednesday and Thursday-morning files of CIC-IDS2017 (1,308,978 flows according to the project documentation; the raw files were not available in the authoring environment, so this count was not recomputed). The sample has 78 numeric features, 11 classes and at least 10 records per class (Table 1); 718 records (71.8%) are benign. Ten features are constant in the sample. The sample was drawn by the project's `prepare_data.py`.

A leakage caveat in the cached sample. The project's original script mean-imputed missing values using all flows *before* sampling. The imputed values in the cached sample therefore carry information from the full dataset. We could not undo this without the raw files; the pipeline below still performs imputation inside each training fold (a no-op on this sample), and the raw-data script `prepare_raw.py` removes the issue for future runs. The practical effect is likely small (a mean over 1.3 M flows) but is not measured.

Table 1: Class distribution of the 1,000-record sample.

Class	Records	%
BENIGN	718	71.8
DoS GoldenEye	17	1.7
DoS Hulk	167	16.7
DoS Slowhttptest	14	1.4
DoS slowloris	14	1.4
FTP-Patator	15	1.5
Heartbleed	10	1.0
SSH-Patator	14	1.4
Web Brute Force	11	1.1
Web Sql Injection	10	1.0
Web XSS	10	1.0

IV Methodology

Pipeline. Fig. 1 shows the protocol. Within each training fold we fit a mean imputer and a standard scaler, then the dimensionality reducer; test folds are only transformed. Feature spaces are: *original* (78 standardised features); *PCA*; and *KPCA* with linear, RBF (scikit-learn default $\gamma = 1/78$) and cosine kernels [7], each with $n \in \{5, 10, 15, 20, 30\}$ components. The original design tied kernel and component count (RBF–5, linear–10, cosine–15); here they are crossed so the two effects can be separated. The reducer is fitted once per fold and shared by all classifiers, which is equivalent to fitting it inside each pipeline because it is deterministic (verified by a unit test against a scikit-learn `Pipeline`).

Classifiers. Logistic regression (L2, lbfgs), k-NN with $k = 5$ [12], an SVM with RBF kernel, $C = 1$ and γ="scale" [11], a decision tree with entropy criterion [13], and a random forest with 100 trees [14], using scikit-learn [10]. Hyper-parameters are fixed to the project's original values and *not* tuned. Each classifier except k-NN is also run with `class_weight=balanced`. A majority-class classifier is the trivial baseline. Resampling (e.g. SMOTE) is not used: some classes have only 7–8 training records per fold, too few for meaningful synthetic interpolation.

Validation. Repeated stratified 5-fold cross-validation, 3 repeats (15 folds), seed 42; every method sees identical folds, giving paired observations. Rare classes have 2–3 test records per fold, so per-fold macro-F1 is noisy; we therefore report means and standard deviations over the 15 folds.

Metrics. Accuracy; macro-precision, macro-recall and macro-F1; weighted-F1; balanced accuracy; and the multi-class Matthews correlation coefficient (MCC) [15]. **Macro-F1 is the primary model-selection criterion**, chosen before the runs because it weights all 11 classes equally. Per-class results and the confusion matrix use pooled out-of-fold predictions of the first repeat.

Statistics. For each classifier we compare feature spaces at a matched $n = 10$ with two-sided Wilcoxon signed-rank tests on the 15 paired macro-F1 values, Holm-corrected over the 30 comparisons, with the matched-pairs rank-biserial correlation r as effect size, and Friedman omnibus tests [16]. Because cross-validation folds share training data, the 15 differences are not independent and the p-values are optimistic [17]; we use them for screening, not as proof. A second family compares the original space with each classifier's best reduced space (selected on the same folds, which favours the reduced space).

Cost. A serial benchmark (no parallel workers) records training and inference time (median of 5 repetitions, one fold split) and peak memory from Python's `tracemalloc`, measured in a separate pass. Native allocations inside compiled code are

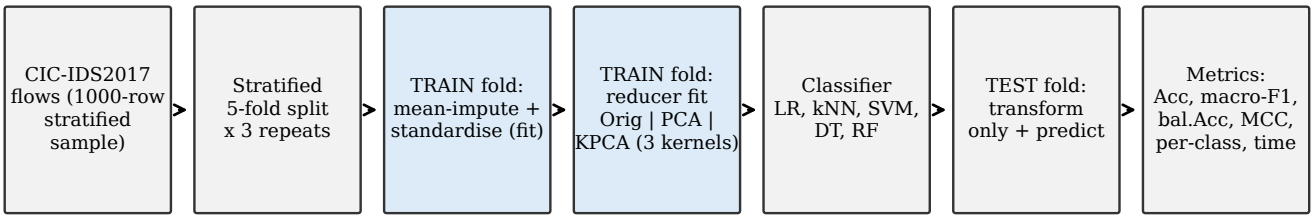


Figure 1: Experimental pipeline. All data-dependent preprocessing and reduction is fitted on the training fold only.

not traced, so memory figures are lower bounds that are most reliable for comparing the reducers.

V Experimental Setup

All experiments ran on a 2-vCPU Intel Xeon @ 2.10 GHz virtual machine with 7 GB RAM (Linux, Python 3.13.16, scikit-learn 1.9.1, NumPy 2.5.3, SciPy 1.18.1). The main grid contains 21 feature spaces $\times$ 5 classifiers $\times$ 2 weighting schemes (plus the baseline) $\times$ 15 folds = 2,850 fits, run with two parallel workers; timings in Table 5 come from the separate serial benchmark. A second run repeats the five main spaces at $n = 10$ with 10 random stratified splits at test fractions 0.2, 0.4 and 0.6 (training size 800, 600, 400) to examine sensitivity to training-set size. This is *not* a temporal or cross-dataset split: the cached sample has no timestamps or day labels.

VI Results

VI.1 Original-feature baseline and the cost of reading accuracy alone

Predicting the majority class gives accuracy 0.718 but macro-F1 0.076 and MCC 0.000. The random forest on original features reaches accuracy 0.952 ± 0.010 but macro-F1 0.722 ± 0.058 (weighted-F1 0.947, balanced accuracy 0.717, MCC 0.894 ± 0.022): the gap between accuracy and macro-F1 comes from rare classes. Per class (Table 2), benign, DoS Hulk, FTP-Patator, SSH-Patator and Heartbleed are detected with F1 $\geq$ 0.96, whereas the three web attacks (brute force, SQL injection, XSS) obtain F1 between 0.21 and 0.50 on only 10–11 records each, and slow-DoS recalls are 0.57–0.71. These are the weak points that accuracy hides (Figs. 2 and 2).

VI.2 Original space versus PCA versus Kernel PCA

Table 3 and Fig. 2 compare spaces at $n = 10$. For every classifier, the original space has the highest or (for k-NN) statistically indistinguishable macro-F1. Of the 30 Holm-corrected comparisons, 17 were significant; in every significant comparison involving the original space it was the better one, and RBF-KPCA was significantly worse than PCA for LR, SVM, DT and RF. Cosine KPCA differed significantly from PCA only for logistic regression (where PCA was better). Against each classifier's *best* reduced space (Table 4), the original features still won significantly for LR, DT and RF (Holm $p < 0.01$, large effects); k-NN showed no difference; and the SVM was the single clear case where reduction helped: cosine KPCA with 30 components raised macro-F1 from 0.388 to 0.510 ($p_{\mathrm{Holm}} = 0.0003$, better in all 15 folds). That comparison is selected on the same folds and therefore optimistic, and the SVM baseline itself is weak (class weighting alone lifts it to 0.546), so we read it as a sign that default-γ RBF-SVM is poorly matched to heavy-tailed standardised features rather than as a general benefit of KPCA.

Table 2: Per-class results of the selected model (RF, original features), pooled out-of-fold predictions, repeat 1.

Class	Precision	Recall	F1	Support
BENIGN	0.965	0.992	0.978	718
DoS GoldenEye	0.875	0.824	0.848	17
DoS Hulk	0.975	0.952	0.964	167
DoS Slowhttptest	1.000	0.571	0.727	14
DoS slowloris	0.909	0.714	0.800	14
FTP-Patator	1.000	1.000	1.000	15
Heartbleed	1.000	1.000	1.000	10
SSH-Patator	0.933	1.000	0.966	14
Web Brute Force	0.250	0.182	0.211	11
Web Sql Injection	0.667	0.400	0.500	10
Web XSS	0.400	0.400	0.400	10

VI.3 Kernel and component count

Linear KPCA is mathematically equivalent to PCA on centred data; the two matched to within numerical noise (macro-F1 differences ≤ 0.011 over all classifier and component settings; Fig. 3). Hence the original project's best result (random forest, "linear KPCA, 10 components") is a PCA result, not evidence for a nonlinear kernel. Component count mattered mainly for logistic regression (macro-F1 0.28 at 5 components to 0.57 at 30 with PCA, still below 0.61 on the original features) and for

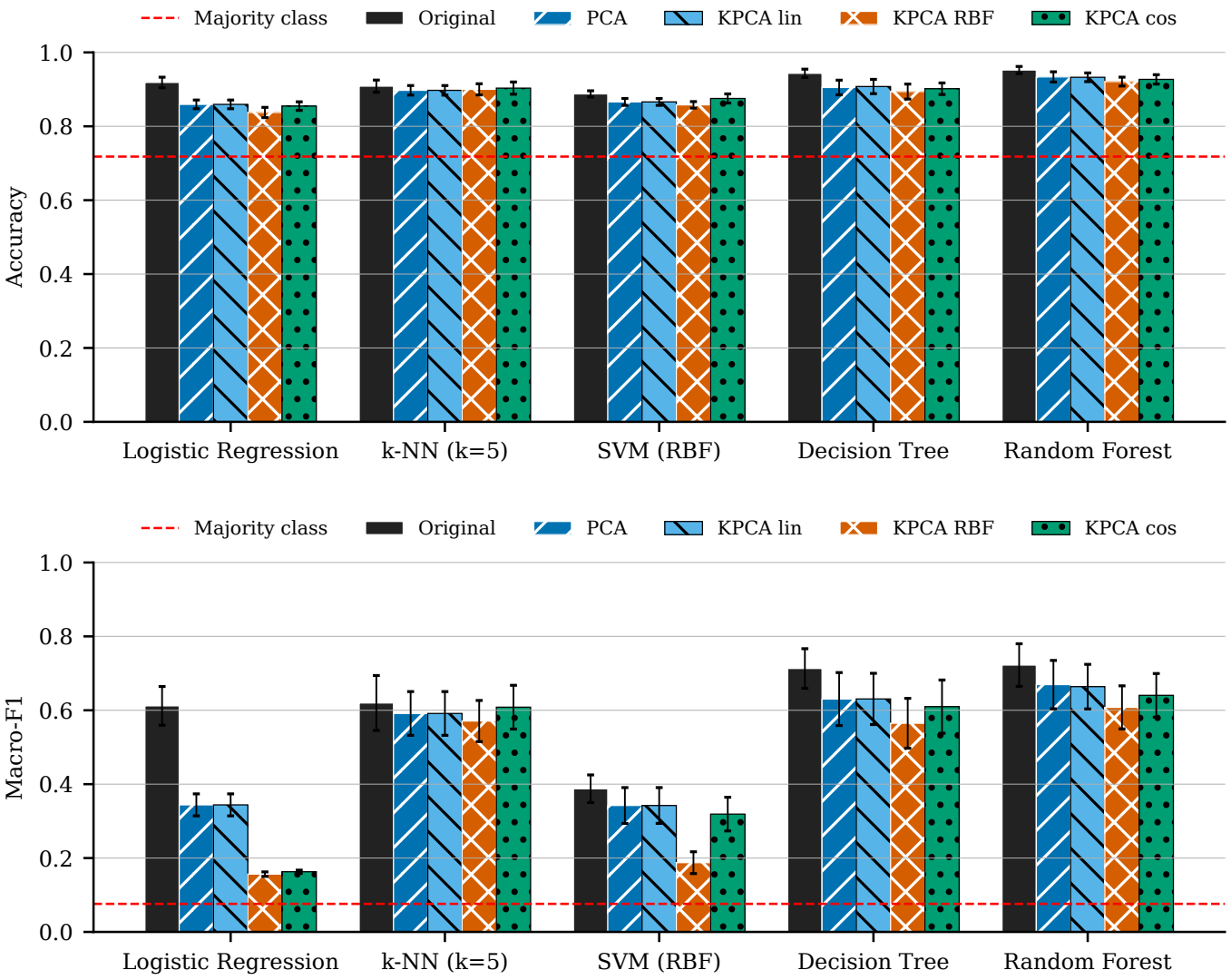


Figure 2: Accuracy (top) and macro-F1 (bottom), mean ± std over 15 folds, 10 components for reduced spaces, standard class weights. Dashed line: majority-class classifier. Axes start at zero.

the cosine-KPCA SVM; for k-NN, DT and RF the curves are flat (Fig. 3). RBF-KPCA with the default γ was the weakest kernel for most classifiers; we did not tune γ, so this is a statement about the default, not about RBF-KPCA in general.

VI.4 Class weighting

Balanced class weights raised balanced accuracy for LR (0.638 $\rightarrow$ 0.778) and SVM (0.358 $\rightarrow$ 0.720) on the original features, but macro-F1 improved only for LR (0.612 $\rightarrow$ 0.630) and SVM (0.388 $\rightarrow$ 0.546) and *fell* for DT and RF (0.713 $\rightarrow$ 0.702, 0.722 $\rightarrow$ 0.687). The best macro-F1 overall nonetheless remained the unweighted random forest, so the selected model does not depend on this choice.

VI.5 Computational cost

Table 5 summarises the serial benchmark at 800 training / 200 test records. KPCA fitting took about 0.09–0.12 s against 0.01 s for PCA, increased decision-tree training time from 0.016 s to roughly 0.09–0.10 s and random-forest training from 0.21 s to 0.36–0.38 s, and roughly doubled random-forest inference time (0.007 s to 0.014–0.016 s per 200 records), since each new sample needs kernel evaluations against all training records. Peak traced memory was 11.7 MB for KPCA against 1.7 MB for PCA or no reduction. Over $n = 400, 600, 800, 1000$ KPCA fit time and memory grew roughly quadratically (3.1, 6.4, 11.0, 16.8 MB; $\approx n^2$), as expected for an $n \times n$ kernel matrix. Extrapolating the memory formula, a kernel matrix for all 1.3 M source flows would need on the order of 14 TB of 64-bit floats, so exact KPCA cannot be applied to the full dataset without ap-

Table 3: Accuracy / macro-F1 (± std over 15 folds) at 10 components (original = 78 features), standard class weights.

Space	LR	KNN	SVM	DT	RF
Original (78)	0.919 / 0.612±0.052	0.909 / 0.620±0.074	0.888 / 0.388±0.037	0.943 / 0.713±0.054	0.952 / 0.722±0.058
PCA-10	0.859 / 0.344±0.030	0.897 / 0.591±0.059	0.866 / 0.342±0.048	0.905 / 0.630±0.072	0.934 / 0.669±0.065
KPCA-lin-10	0.859 / 0.344±0.030	0.897 / 0.591±0.059	0.866 / 0.342±0.048	0.908 / 0.631±0.069	0.933 / 0.664±0.060
KPCA-RBF-10	0.837 / 0.156±0.006	0.900 / 0.571±0.056	0.858 / 0.187±0.029	0.894 / 0.565±0.067	0.921 / 0.608±0.058
KPCA-cos-10	0.855 / 0.163±0.004	0.903 / 0.608±0.059	0.875 / 0.319±0.046	0.902 / 0.610±0.072	0.927 / 0.640±0.059

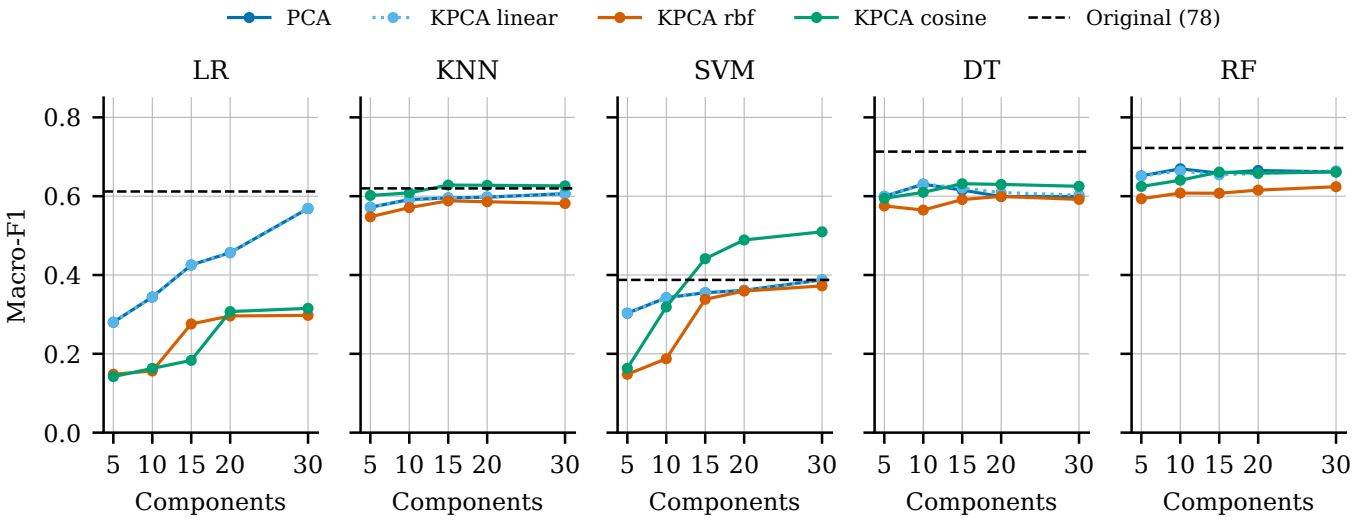


Figure 3: Macro-F1 versus number of components for PCA and three KPCA kernels; dashed line: original 78 features.

Table 4: Original features versus each classifier's best reduced space (Wilcoxon, Holm-corrected). Positive Δ: original better.

Clf	Best reduced space	F1 orig	F1 reduced	ΔF1 (orig-red.)	p_{Holm}	r
LR	kpca-linear-30	0.612	0.569	0.043	0.000	1.000
KNN	kpca-cosine-15	0.620	0.628	-0.008	0.679	-0.133
SVM	kpca-cosine-30	0.388	0.510	-0.122	0.000	-1.000
DT	kpca-cosine-15	0.713	0.632	0.082	0.001	0.933
RF	pca-10	0.722	0.669	0.053	0.002	0.981

Table 5: Serial runtime and traced peak memory (800 train / 200 test), median of 5.

Space	Reducer fit (s)	DT train (s)	RF train (s)	RF infer. (s)	Peak mem (MB)
Original (78)	0.000	0.018	0.190	0.007	1.687
PCA-10	0.010	0.016	0.208	0.007	1.686
KPCA-lin-10	0.104	0.085	0.370	0.014	11.750
KPCA-RBF-10	0.119	0.100	0.359	0.015	11.750
KPCA-cos-10	0.091	0.098	0.375	0.016	11.750

proximation (not tested here). Absolute times are sub-second on a shared 2-vCPU machine; differences below roughly a factor of two should not be over-interpreted. We make no real-time claim: only batch latencies on this machine were measured.

VI.6 Sensitivity to training-set size

Fig. 4 shows macro-F1 at 400, 600 and 800 training records over 10 random splits each. For LR, SVM, DT and RF the original space had the highest mean macro-F1 at every size, so the ranking in Table 3 is not an artefact of one training size; for k-NN the spaces overlap within one standard deviation. This is a stability check within the same sample and distribution, not a test of generalization to unseen traffic.

VII Discussion

Answers to the research questions. RQ1/RQ2: on this sample, reducing 78 standardised features to 5–30 PCA or KPCA components did not improve and usually lowered macro-F1; the one robust gain was for the RBF-SVM with cosine KPCA. RQ3: kernel mattered more than component count for most classifiers, with RBF (default γ) worst; linear KPCA equals PCA. RQ4: KPCA multiplied fitting time about tenfold and memory about sevenfold at $n = 800$ and scales quadratically, with no accuracy benefit to offset it for four of five classifiers. RQ5: orderings were stable across repeated folds and training sizes within the sample.

Why accuracy misleads. The benign class makes up 71.8% of records, so the trivial baseline already has accuracy 0.718; high accuracies of 0.93–0.95 for the best models correspond to macro-F1 of 0.66–0.72. The original project reported weighted-F1 and accuracy from single splits, which hide rare-class failure and cannot show whether a difference of a point or two is noise

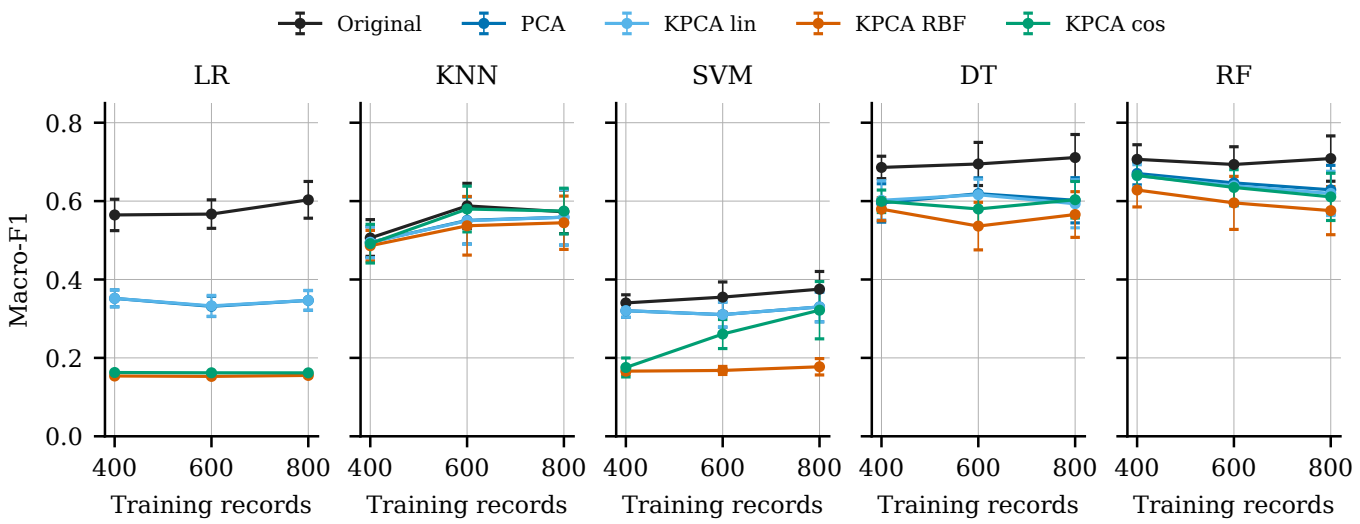


Figure 4: Macro-F1 versus training-set size (random stratified splits, 10 per size, mean ± std); 10 components for reduced spaces.

(the standard deviation of macro-F1 across folds is 0.05–0.07).

Plausible but untested explanations. Tree ensembles are insensitive to monotone feature scaling and can exploit individual informative features; projecting onto a few leading components may discard low-variance but discriminative directions. Heavy-tailed flow features also make variance-based reduction and the default RBF width poorly matched. These are hypotheses we did not test (e.g. with tuned γ, robust scaling, or supervised reduction).

VIII Threats to Validity and Limitations

Sample size and imbalance. The sample has 1,000 records; nine of the ten attack classes have only 10–17 records, so per-class estimates are very uncertain and the macro-F1 is sensitive to a few records. **Population.** Only three of the eight CIC-IDS2017 files were sampled, one sample was used, and each attack family occurs on a single day in those files. **Dataset quality.** The original dataset contains known labelling and feature artefacts [3], which may inflate or distort scores. **Residual leakage.** The cached sample's imputation used full-data means (Section III). **Selection and inference.** The model was selected on the same cross-validation that produced the reported scores, so its scores are optimistic; no external hold-out was possible with 10-record classes. Repeated-CV tests are not independent [17]. **Tuning.** Hyper-parameters, including the RBF γ, were fixed, not tuned. **Generalization.** No temporal, cross-day or cross-dataset evaluation was performed [9]; conclusions apply to random splits of this sample. **Cost.** Memory is a traced lower bound; timings are from one virtual machine. We therefore make no claim of state-of-the-art, real-time or deployment-ready performance.

IX Conclusion

On a 1,000-record sample of CIC-IDS2017 and under leakage-controlled repeated cross-validation, a random forest on the original 78 features gave the best class-aware performance (macro-F1 0.722 ± 0.058, accuracy 0.952 ± 0.010). PCA and Kernel PCA with 5–30 components did not improve on the original features for logistic regression, decision tree or random forest, made no significant difference for k-NN, and helped only the RBF-SVM (cosine KPCA, 30 components), at a roughly tenfold fitting-time and sevenfold memory cost with quadratic scaling. Linear KPCA coincided with PCA, so previously reported "KPCA" gains with a linear kernel are linear-PCA gains. Accuracy alone concealed weak detection of web attacks. The results are limited to this sample and to random splits.

Future work. Rerun with the full or larger CIC-IDS2017 subsets using the provided `prepare_raw.py`; add chronological within-day and cross-dataset evaluation; tune γ and use robust scaling; test the corrected CIC-IDS2017 release [3]; and evaluate approximate KPCA for scalability.

Reproducibility

Code, per-fold raw results and instructions are in the project repository (`RUN_EXPERIMENTS.md`); the main grid runs in a few minutes on two cores. Data come from CIC-IDS2017 [2].

References

[1] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Comput. Secur.*, 2019, arXiv:1903.02460.

[2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[3] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12, doi:10.1109/SPW53761.2021.00009.

[4] R. Panigrahi and S. Borah, "A detailed analysis of CICIDS2017 dataset for designing intrusion detection systems," *Int. J. Eng. Technol.*, vol. 7, no. 3.24, pp. 479–482, 2018.

[5] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 3971–3988.

[6] R. Abdulhammed, H. Musafer, A. Alessa, M. Faezipour, and A. Abuzneid, "Features dimensionality reduction approaches for machine learning based network intrusion detection," *Electronics*, vol. 8, no. 3, Art. no. 322, 2019.

[7] B. Schölkopf, A. Smola, and K.-R. Müller, "Nonlinear component analysis as a kernel eigenvalue problem," *Neural Comput.*, vol. 10, no. 5, pp. 1299–1319, 1998.

[8] F. Kuang, W. Xu, and S. Zhang, "A novel hybrid KPCA and SVM with GA model for intrusion detection," *Appl. Soft Comput.*, vol. 18, pp. 178–184, 2014.

[9] G. Apruzzese, L. Pajola, and M. Conti, "The cross-evaluation of machine learning-based network intrusion detection systems," *IEEE Trans. Netw. Service Manag.*, 2022, doi:10.1109/TNSM.2022.3157344.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[12] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[13] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986.

[14] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[15] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, Art. no. 6, 2020.

[16] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.

[17] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Mach. Learn.*, vol. 52, no. 3, pp. 239–281, 2003.